5104 Assignment 1

Hope Mullins

2025-02-14

Begin by installing and loading the necessary packages:

```
# install.packages('rjson')
# install.packages("jsonlite")
# install.packages("rhdf5")
# install.packages("MASS")
# install.packages("fitdistrplus")
# install.packages("Magick")

library(rjson)
library(jsonlite)
```

```
##
## Attaching package: 'jsonlite'

## The following objects are masked from 'package:rjson':
##
##     fromJSON, toJSON
```

```
library(rhdf5)
library(MASS)
library(fitdistrplus)
```

```
## Loading required package: survival
```

Question 1

1.

Load the Iris dataset. Note that you will have to choose it yourself.

```
iris <- read.csv(file.choose(), sep = ',')
```

2.

Next, save the dataset to a JSON file.

```r
write_json(iris, "iris.json", pretty = TRUE, dataframe = "columns")
```

3.

Proceed by loading the data back from JSON.

```r
iris_from_json <- fromJSON("iris.json")
iris_from_json <- as.data.frame(iris_from_json)
```

4.

Now, let's store the data we loaded from JSON into HDF5.

```r
h5createFile("iris.h5")
h5createGroup("iris.h5", "irises")

for (i in names(iris_from_json)) {
  h5write(iris_from_json[[i]], "iris.h5", paste0("irises/", i))
}

h5ls("iris.h5")
```

```
##     group         name       otype dclass dim
## 0       /       irises   H5I_GROUP
## 1 /irises petal.length H5I_DATASET  FLOAT 150
## 2 /irises  petal.width H5I_DATASET  FLOAT 150
## 3 /irises sepal.length H5I_DATASET  FLOAT 150
## 4 /irises  sepal.width H5I_DATASET  FLOAT 150
## 5 /irises      variety H5I_DATASET STRING 150
```

5.

Without loading in the entire dataset, we may randomly load in a particular number of rows (10). We need to first establish how many rows there are.

```r
num_rows <- length(h5read("iris.h5", "irises/petal.length"))
```

Now, let's proceed with creating the 10-row dataframe.

```r
set.seed(12345)
iris_ten <- data.frame(
  p_length = h5read("iris.h5", "irises/petal.length", index = list(sample(1:num_rows, 10))),
  p_width = h5read("iris.h5", "irises/petal.width", index = list(sample(1:num_rows, 10))),
  s_length = h5read("iris.h5", "irises/sepal.length", index = list(sample(1:num_rows, 10))),
  s_width = h5read("iris.h5", "irises/sepal.width", index = list(sample(1:num_rows, 10))),
  variety = h5read("iris.h5", "irises/variety", index = list(sample(1:num_rows, 10)))
)

iris_ten
```

```
##    p_length p_width s_length s_width    variety
## 1       5.1     2.1      6.3     2.7     Setosa
## 2       4.7     1.0      4.3     3.0 Versicolor
## 3       3.3     0.1      6.7     3.4 Versicolor
## 4       4.0     0.2      7.6     3.4 Versicolor
## 5       4.3     0.1      7.7     3.2  Virginica
## 6       4.2     0.2      5.7     3.4  Virginica
## 7       1.4     0.2      5.7     2.8 Versicolor
## 8       4.5     1.3      5.9     2.8  Virginica
## 9       5.2     0.2      6.2     2.4     Setosa
## 10      1.4     0.2      7.7     3.0  Virginica
```

Using this particular seed, our dataframe returns two rows with the Setosa species, four with the Versicolor species, and four of the Virginica species.

Question 2

1.

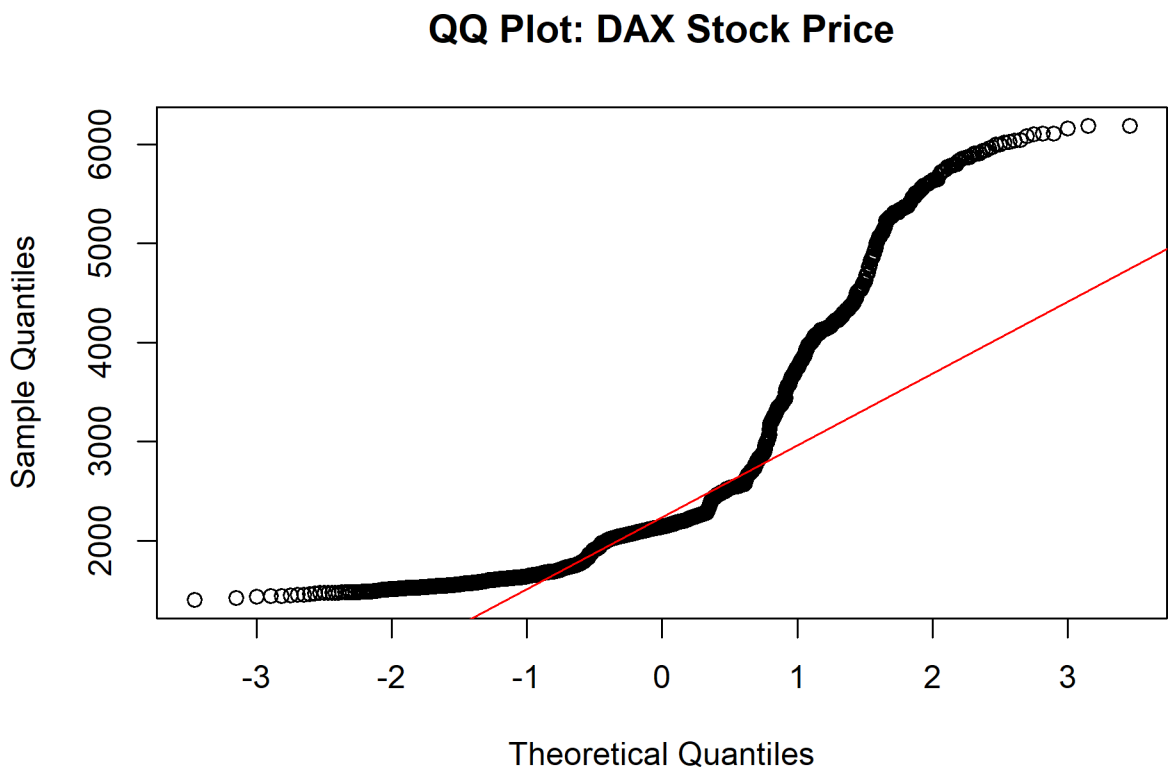
Load the EUStockMarkets dataset. Note that you will have to choose it yourself.

```r
EUSM <- read.csv(file.choose(), sep = ',')
```

2.

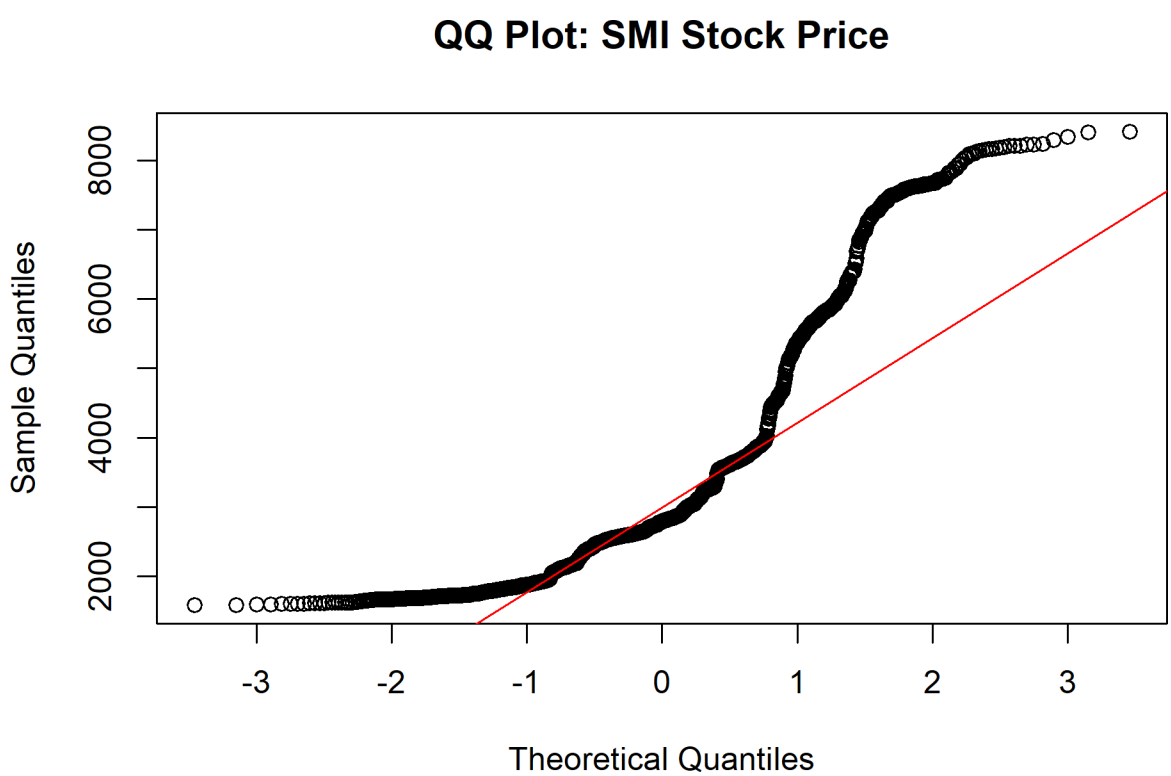
Using this data, produce a QQplot for each stock price. Starting with DAX:

```r
qqnorm(EUSM$DAX, main = "QQ Plot: DAX Stock Price")
qqline(EUSM$DAX, col = "red")
```



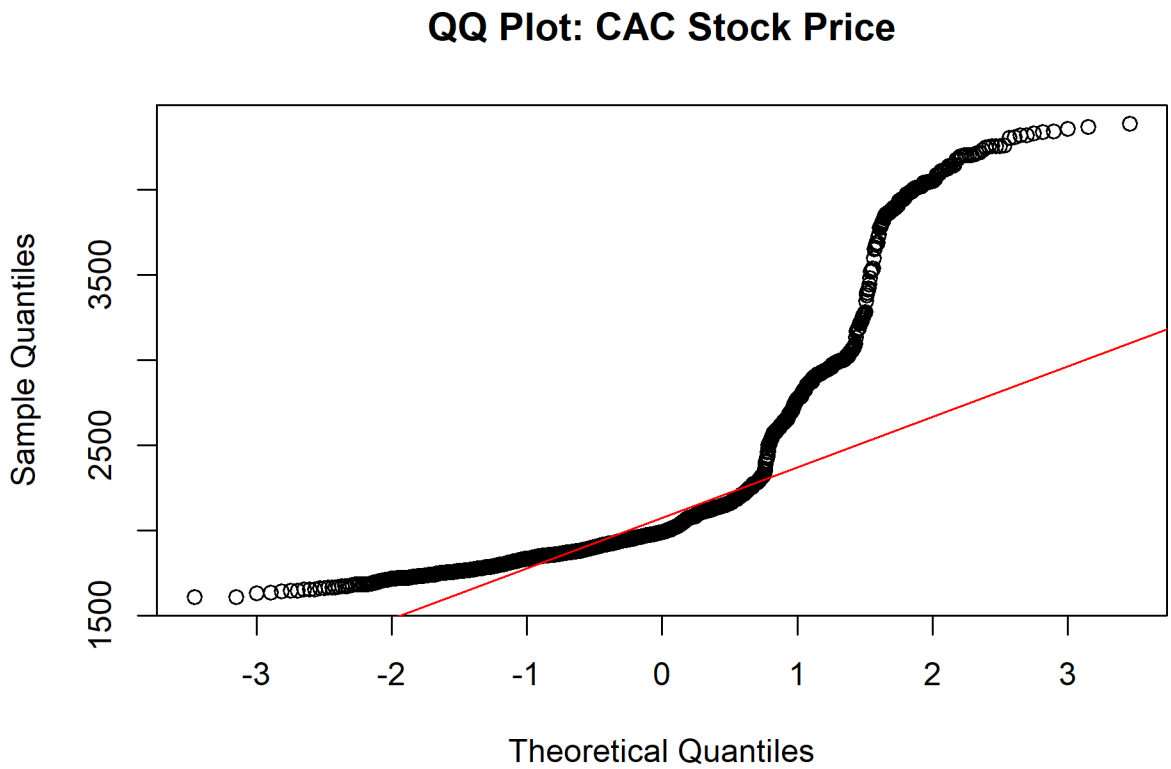
For SMI:

```
qqnorm(EUSM$SMI, main = "QQ Plot: SMI Stock Price")
qqline(EUSM$SMI, col = "red")
```



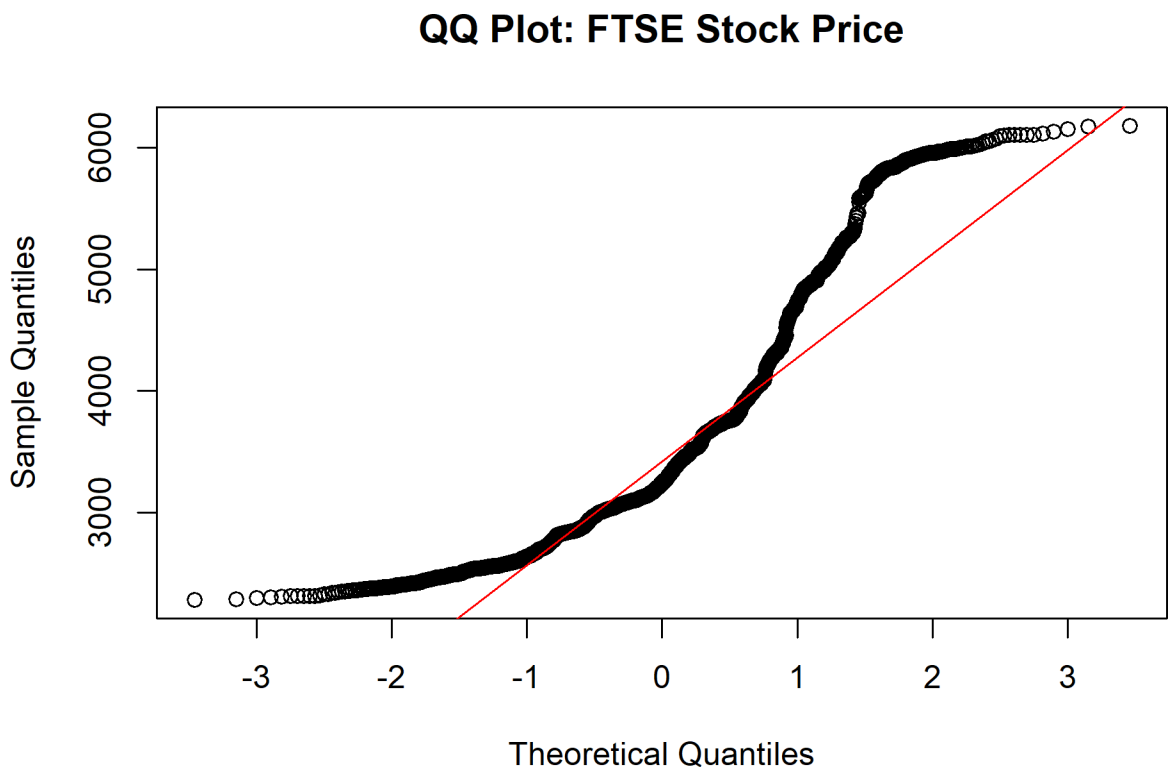
For CAC:

```r
qqnorm(EUSM$CAC, main = "QQ Plot: CAC Stock Price")
qqline(EUSM$CAC, col = "red")
```



For FTSE:

```r
qqnorm(EUSM$FTSE, main = "QQ Plot: FTSE Stock Price")
qqline(EUSM$FTSE, col = "red")
```



When comparing these QQ plots to their respective lines, none of them follow the lines very closely. This demonstrates that none of the stock prices are normally distributed.

3.

Now, let's produce a KS test of normality for each stock price. We need to standardize our data.

```
ks.test(scale(EUSM$DAX), "pnorm")
```

```
## Warning in ks.test.default(scale(EUSM$DAX), "pnorm"): ties should not be
## present for the one-sample Kolmogorov-Smirnov test

##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  scale(EUSM$DAX)
## D = 0.2185, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```
ks.test(scale(EUSM$SMI), "pnorm")
```

```
## Warning in ks.test.default(scale(EUSM$SMI), "pnorm"): ties should not be
## present for the one-sample Kolmogorov-Smirnov test

##
##  Asymptotic one-sample Kolmogorov-Smirnov test
```

```
##
## data:  scale(EUSM$SMI)
## D = 0.17193, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```r
ks.test(scale(EUSM$CAC), "pnorm")
```

```
## Warning in ks.test.default(scale(EUSM$CAC), "pnorm"): ties should not be
## present for the one-sample Kolmogorov-Smirnov test
```

```
##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  scale(EUSM$CAC)
## D = 0.23759, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```r
ks.test(scale(EUSM$FTSE), "pnorm")
```

```
## Warning in ks.test.default(scale(EUSM$FTSE), "pnorm"): ties should not be
## present for the one-sample Kolmogorov-Smirnov test
```

```
##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  scale(EUSM$FTSE)
## D = 0.13156, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

Because duplicate values violate the requirements of performing the KS test, and we have duplicate values in each of our price columns even after standardizing the data, we should try jittering the data.

```r
ks.test(jitter(scale(EUSM$DAX)), "pnorm")
```

```
##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  jitter(scale(EUSM$DAX))
## D = 0.21849, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```r
ks.test(jitter(scale(EUSM$SMI)), "pnorm")
```

```
##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  jitter(scale(EUSM$SMI))
## D = 0.17197, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```r
ks.test(jitter(scale(EUSM$CAC)), "pnorm")
```

```
##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  jitter(scale(EUSM$CAC))
## D = 0.23765, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

```r
ks.test(jitter(scale(EUSM$FTSE)), "pnorm")
```

```
##
##  Asymptotic one-sample Kolmogorov-Smirnov test
##
## data:  jitter(scale(EUSM$FTSE))
## D = 0.13156, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

Based on these tests, all of the p-values are the lowest they can be using the KS test of normality in R, further confirming that the stock prices are likely not normal.

Additionally, for fun, we might attempt using the Shapiro-Wilk test considering it is a pretty robust option:

```r
shapiro.test(EUSM$DAX)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  EUSM$DAX
## W = 0.79943, p-value < 2.2e-16
```

```r
shapiro.test(EUSM$SMI)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  EUSM$SMI
## W = 0.83826, p-value < 2.2e-16
```

```r
shapiro.test(EUSM$CAC)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  EUSM$CAC
## W = 0.73914, p-value < 2.2e-16
```

```r
shapiro.test(EUSM$FTSE)
```

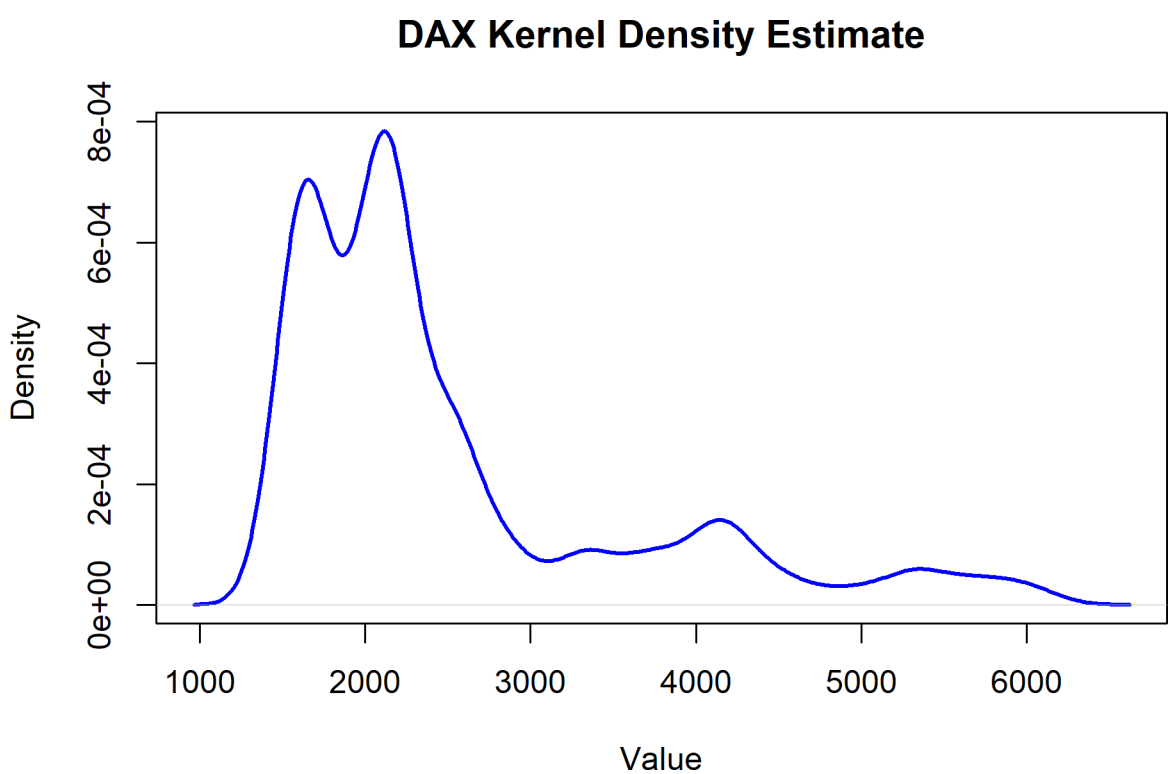
```
##
##  Shapiro-Wilk normality test
##
## data:  EUSM$FTSE
## W = 0.88909, p-value < 2.2e-16
```

Altogether, the QQ plots, the KS test and the shapiro test confirm that our stock price data is not normally distributed. The p-values for all the tests are the lowest values they can be, so considering it is below .05, we reject the null hypothesis. We have sufficient evidence to say none of the stock prices come from the normal distribution.

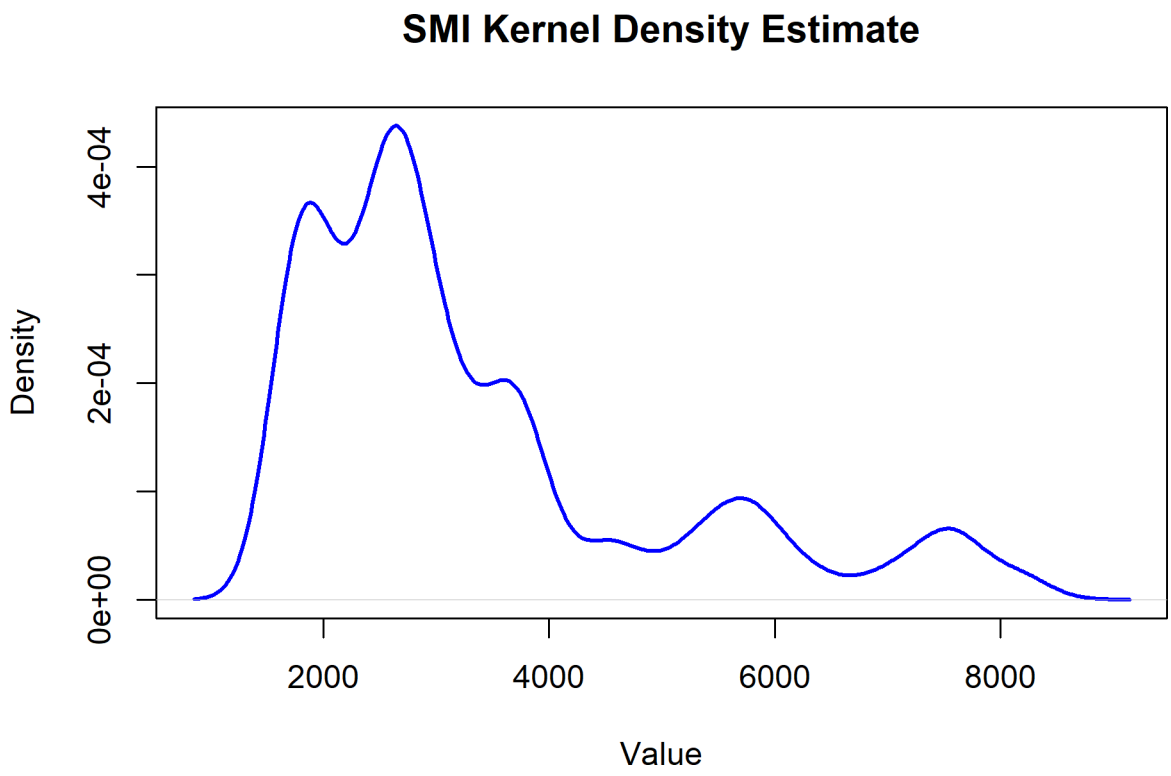
4.

Next, lets create a kernel density plot for each stock price distribution.

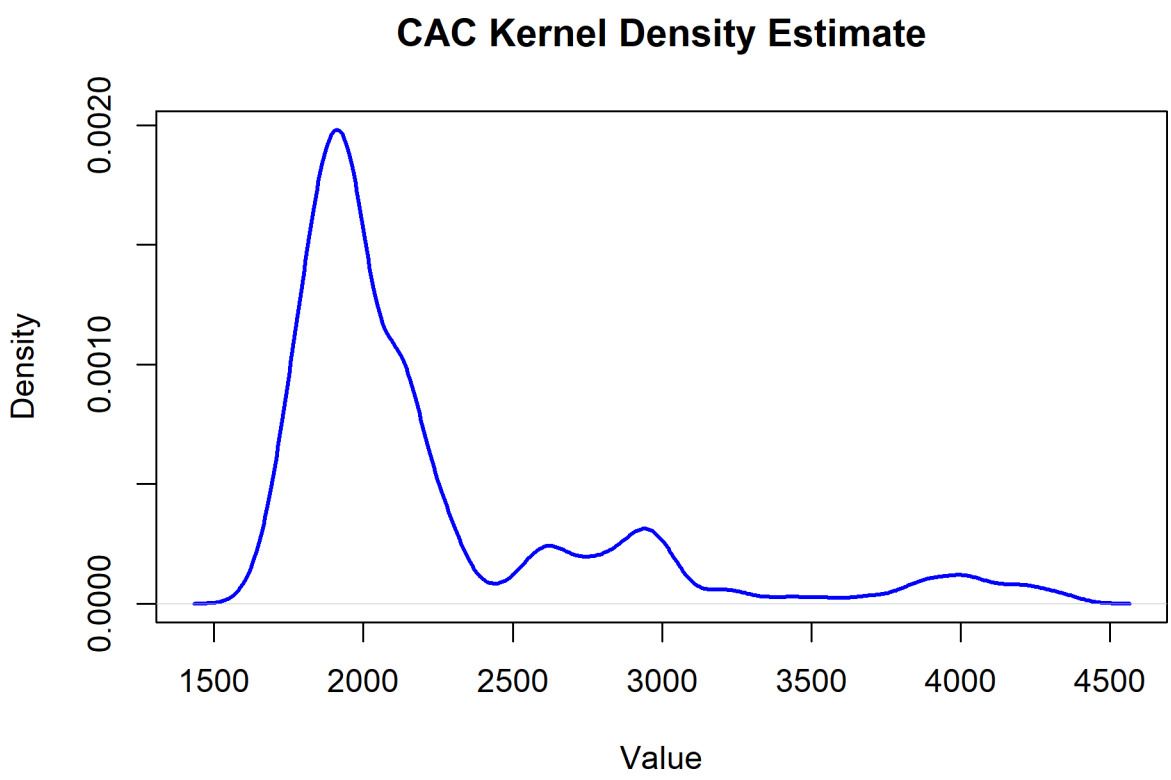
```r
d1 <- density(EUSM$DAX, bw = "nrd0", kernel = "gaussian")
d2 <- density(EUSM$SMI, bw = "nrd0", kernel = "gaussian")
d3 <- density(EUSM$CAC, bw = "nrd0", kernel = "gaussian")
d4 <- density(EUSM$FTSE, bw = "nrd0", kernel = "gaussian")

plot(d1, main = "DAX Kernel Density Estimate", xlab = "Value",
     ylab = "Density", col = "blue", lwd = 2)
```



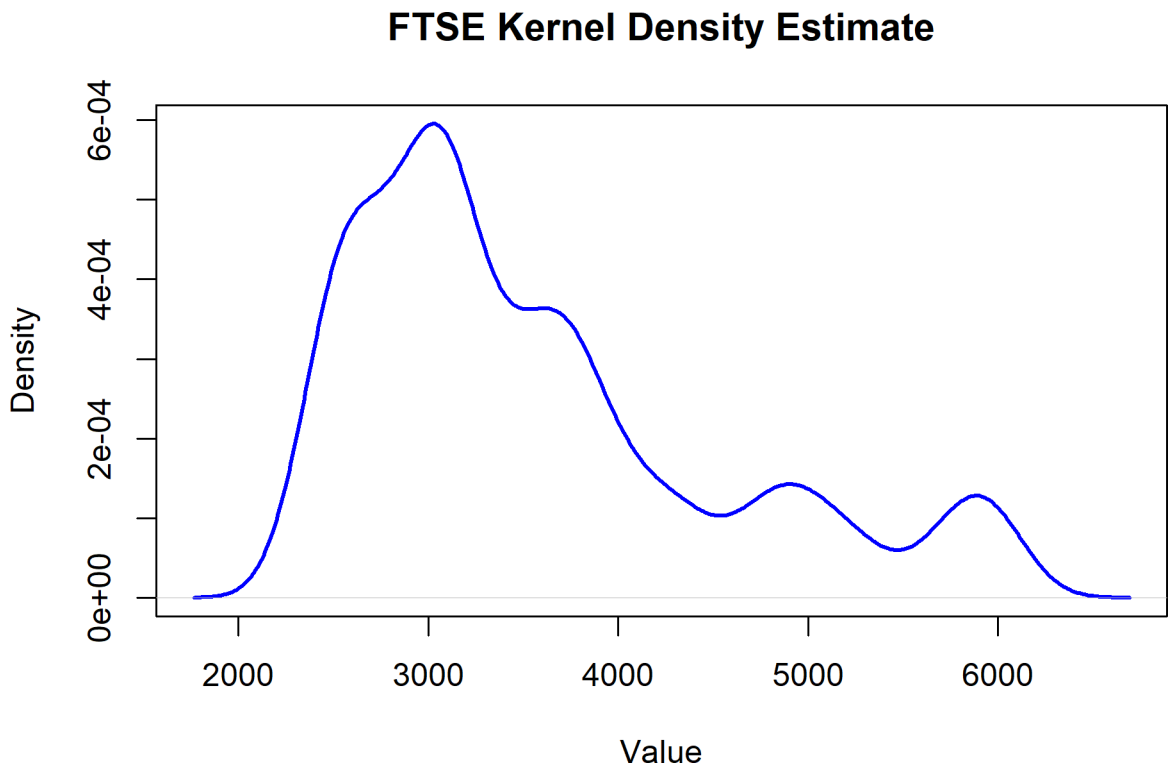

```r
plot(d2, main = "SMI Kernel Density Estimate", xlab = "Value",
     ylab = "Density", col = "blue", lwd = 2)
```




```
plot(d3, main = "CAC Kernel Density Estimate", xlab = "Value",
     ylab = "Density", col = "blue", lwd = 2)
```



```r
plot(d4, main = "FTSE Kernel Density Estimate", xlab = "Value",
     ylab = "Density", col = "blue", lwd = 2)
```



The Kernel density plots for each stock price verify that our stock price data is not normally distributed. There is a very heavy right skew on each of the distributions, and the multiple fluctuations reveal that our data is most likely multi-modal, made up of various distributions instead of a single normal distribution.

Question 3

1.

Load the Anscombe dataset. Note that you will have to choose it yourself.

```r
Anscombe <- read.csv(file.choose(), sep = ',')
```

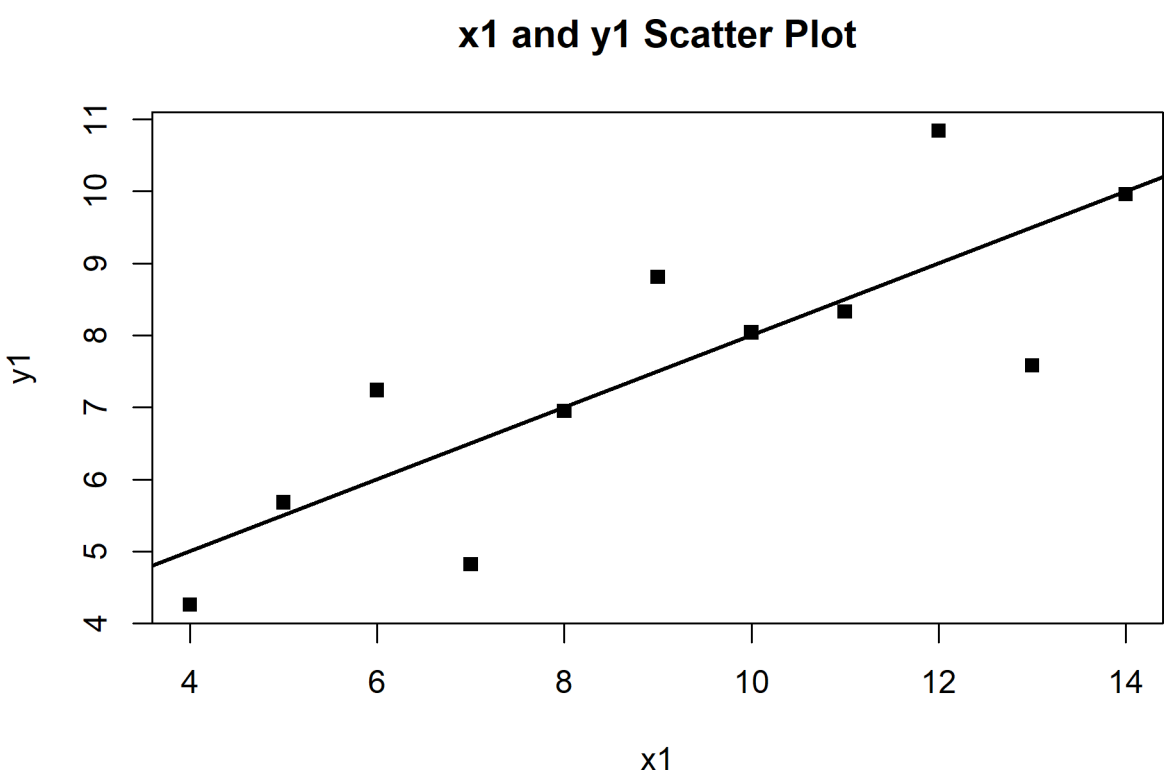
2.

Next, let's plot x1 and y1, and then produce a linear regression.

```r
plot(Anscombe$x1, Anscombe$y1,
     main = "x1 and y1 Scatter Plot",
     xlab = "x1", ylab = "y1", pch = 15)

model1 <- lm(y1 ~ x1, Anscombe)
summary(model1)
```

```
##
## Call:
## lm(formula = y1 ~ x1, data = Anscombe)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.92127 -0.45577 -0.04136  0.70941  1.83882
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.0001     1.1247   2.667  0.02573 *
## x1            0.5001     0.1179   4.241  0.00217 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.237 on 9 degrees of freedom
## Multiple R-squared:  0.6665, Adjusted R-squared:  0.6295
## F-statistic: 17.99 on 1 and 9 DF,  p-value: 0.00217
```

```r
abline(model1, lwd = 2)
```



Generating a model where x1 is the independent variable and y1 is the dependent variable, we end up with an intercept of 3.0001 with an error of 1.1247. This result is statistically significant at the .05 significance level.

The slope of this model is 0.5001 with an error of .1179, and it is statistially significant at the .01 significance level. Also, the R-squared value is .6665, telling us that the variable x1 explains approximately 66.65% of the variation in y1.

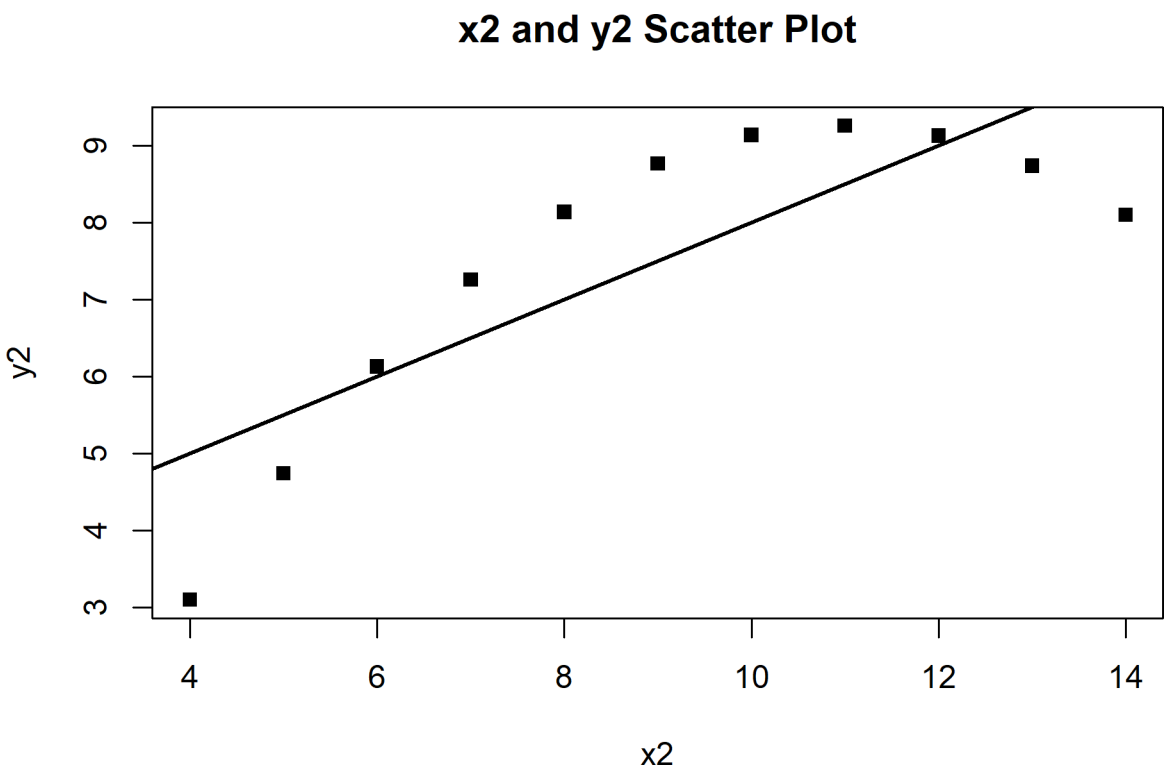
3.

Now, let's repeat this process for x2 and y2.

```r
plot(Anscombe$x2, Anscombe$y2,
     main = "x2 and y2 Scatter Plot",
     xlab = "x2", ylab = "y2", pch = 15)

model2 <- lm(y2 ~ x2, Anscombe)
summary(model2)
```

```
##
## Call:
## lm(formula = y2 ~ x2, data = Anscombe)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.9009 -0.7609  0.1291  0.9491  1.2691
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)    3.001      1.125   2.667  0.02576 *
## x2             0.500      0.118   4.239  0.00218 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.237 on 9 degrees of freedom
## Multiple R-squared:  0.6662, Adjusted R-squared:  0.6292
## F-statistic: 17.97 on 1 and 9 DF,  p-value: 0.002179
```

```r
abline(model2, lwd = 2)
```



Generating a model where x2 is the independent variable and y2 is the dependent variable, we end up with an intercept of 3.001 (just barely greater than model1) with an error of 1.125 (also greater than model1). This result is statistically significant at the .05 significance level as well.

The slope of this model is 0.500 with an error of .118, and it is statistially significant at the .01 significance level. Also, the R-squared value is .6662, telling us that the variable x2 explains approximately 66.62% of the variation in y2.

Observing the scatter plot, we can very clearly see that the relationship between x2 and y2 is not linear.

4.

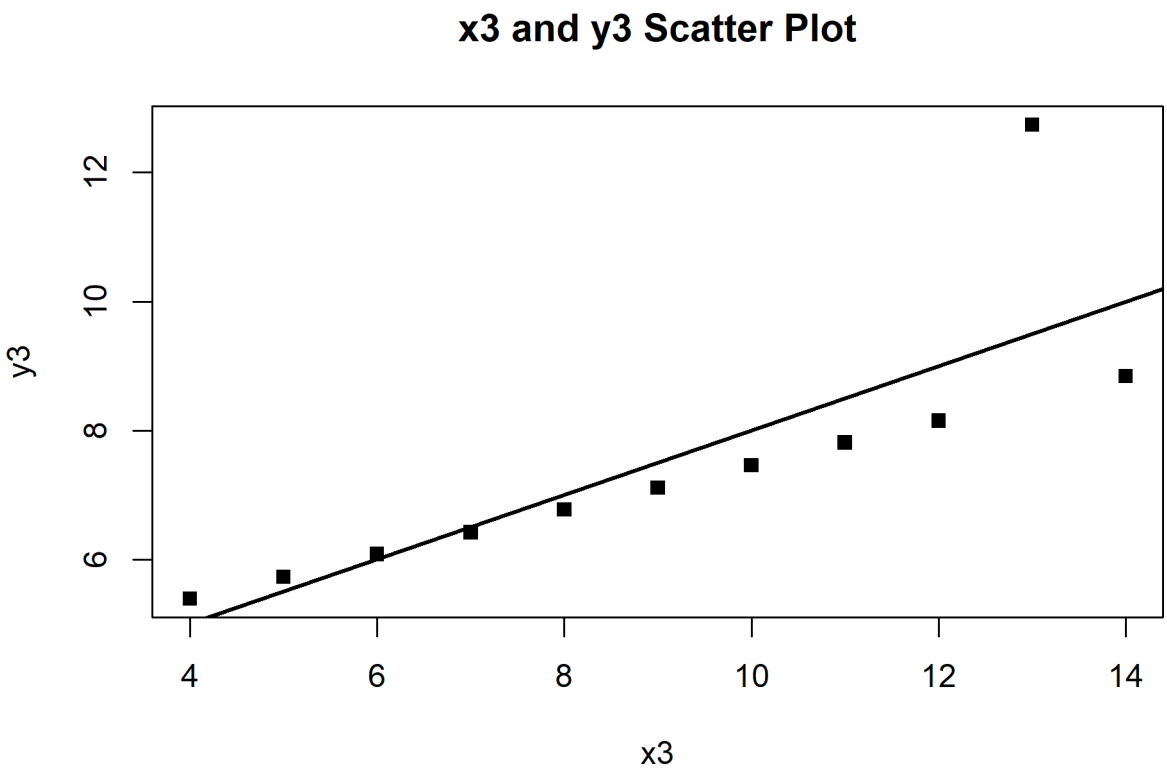
Now, let's repeat this process for x3 and y3.

```
plot(Anscombe$x3, Anscombe$y3,
     main = "x3 and y3 Scatter Plot",
     xlab = "x3", ylab = "y3", pch = 15)

model3 <- lm(y3 ~ x3, Anscombe)
summary(model3)
```

```
##
## Call:
## lm(formula = y3 ~ x3, data = Anscombe)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.1586 -0.6146 -0.2303  0.1540  3.2411
##
```

```
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.0025     1.1245   2.670  0.02562 *
## x3            0.4997     0.1179   4.239  0.00218 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.236 on 9 degrees of freedom
## Multiple R-squared:  0.6663, Adjusted R-squared:  0.6292
## F-statistic: 17.97 on 1 and 9 DF,  p-value: 0.002176
```

```r
abline(model3, lwd = 2)
```



Generating a model where x3 is the independent variable and y3 is the dependent variable, we end up with an intercept of 3.0025 with an error of 1.145. This result is statistically significant at the .05 significance level, as it was for our first two models.

The slope of this model is 0.4997 with an error of .1179, and it is statistically significant at the .01 significance level. Also, the R-squared value is .6663, telling us that the variable x3 explains approximately 66.63% of the variation in y3.

Observing the scatter plot, the relationship between x3 and y3 appears to be much more linear. There is a single outlier, but excluding that value, the slope appears relatively constant. This can be visualized by plotting our model.

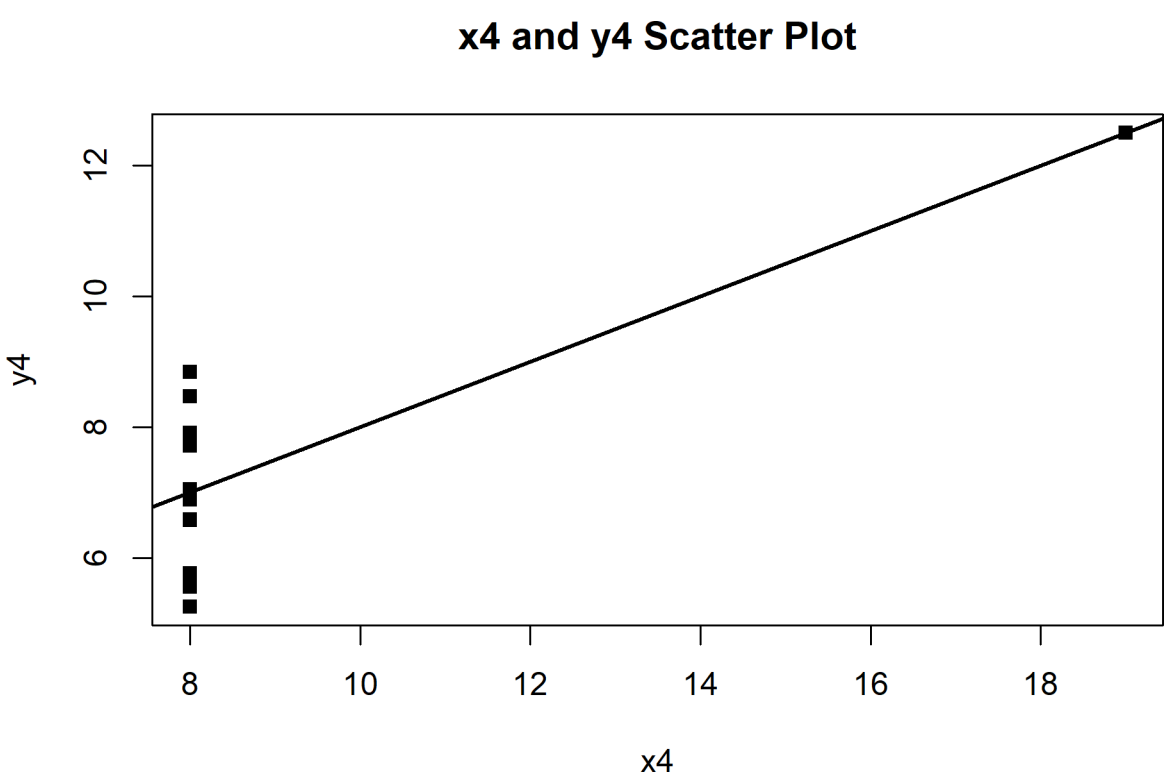
5.

Now, let's repeat this process for x4 and y4.

```r
plot(Anscombe$x4, Anscombe$y4,
     main = "x4 and y4 Scatter Plot",
     xlab = "x4", ylab = "y4", pch = 15)

model4 <- lm(y4 ~ x4, Anscombe)
summary(model4)
```

```
##
## Call:
## lm(formula = y4 ~ x4, data = Anscombe)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -1.751 -0.831  0.000  0.809  1.839
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.0017     1.1239   2.671  0.02559 *
## x4            0.4999     0.1178   4.243  0.00216 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.236 on 9 degrees of freedom
## Multiple R-squared:  0.6667, Adjusted R-squared:  0.6297
## F-statistic:    18 on 1 and 9 DF,  p-value: 0.002165
```

```r
abline(model4, lwd = 2)
```



Generating a model where x4 is the independent variable and y4 is the dependent variable, we end up with an intercept of 3.0017 with an error of 1.1239. This result is statistically significant at the .05 significance level, as it was for our first three models.

The slope of this model is 0.4999 with an error of .1178, and it is statistically significant at the .01 significance level. Also, the R-squared value is .6667, telling us that the variable x4 explains approximately 66.67% of the variation in y4.

Observing the scatter plot, the relationship between x4 and y4 is almost a completely vertical line. As with model4, there is a single outlier, but excluding that point, the slope would be equivalent to positive infinity.

6.

Overall, there are many conclusions we can draw from this experience. All of the relationships plotted are drastically different. x1 and y1 results in a true scatter plot. x2 and y2 is a quadratic relationship. x3 and y3 appears to be mostly linear. x4 and y4 does not result in a function, but rather every y-value besides one corresponds to the same x-value.

Another interesting conclusion to draw is that, despite the visual difference in each of these relationships, the linear regression is almost identical for all four relationships. The slope, intercept, errors, and R-squared values are all extremely close; very minimal differences occur.

Question 4

1.

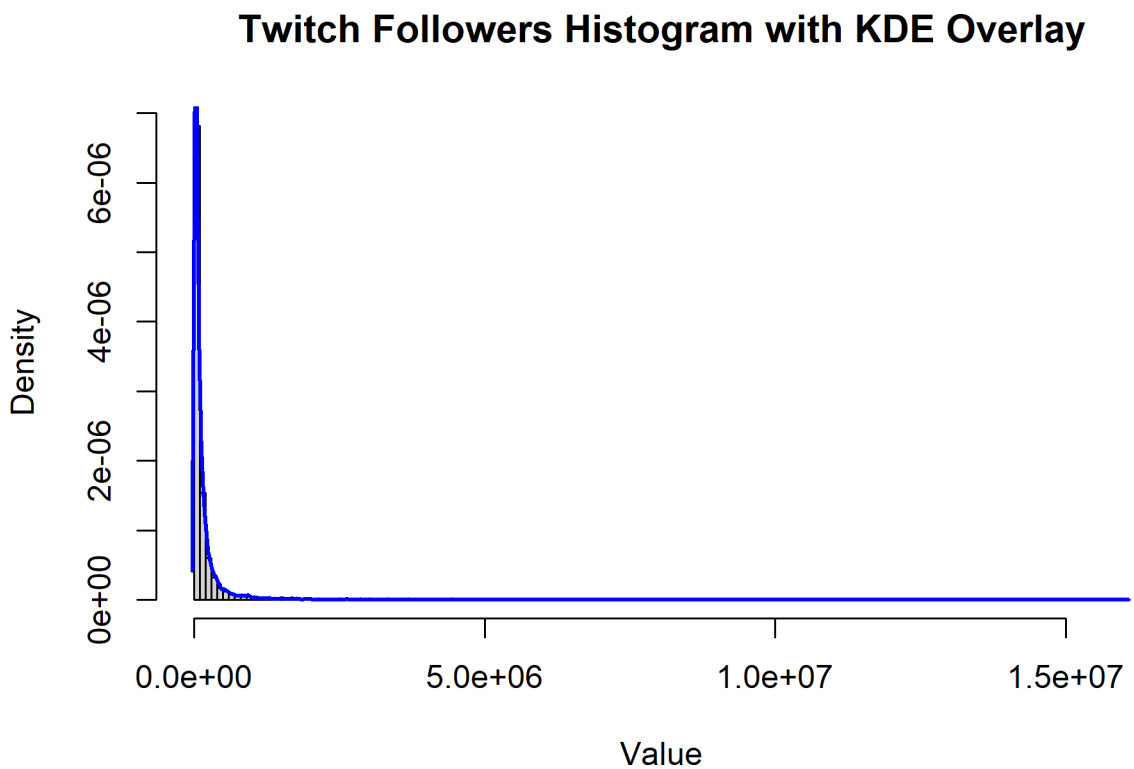
Load the Twitch dataset. Note that you will have to choose it yourself.

```
twitch <- read.csv(file.choose(), sep = ',')
```

2.

Now, let's visualize the data for Twitch followers with a histogram and a kernel density.

```
foll_dens = density(twitch$followers, bw = "nrd0", kernel = "gaussian")

hist(twitch$followers, breaks = 200, prob = T, border = 'black',
     main = "Twitch Followers Histogram with KDE Overlay",
     xlab = "Value", ylab = "Density")
lines(foll_dens, col = "blue", lwd = 2)
```

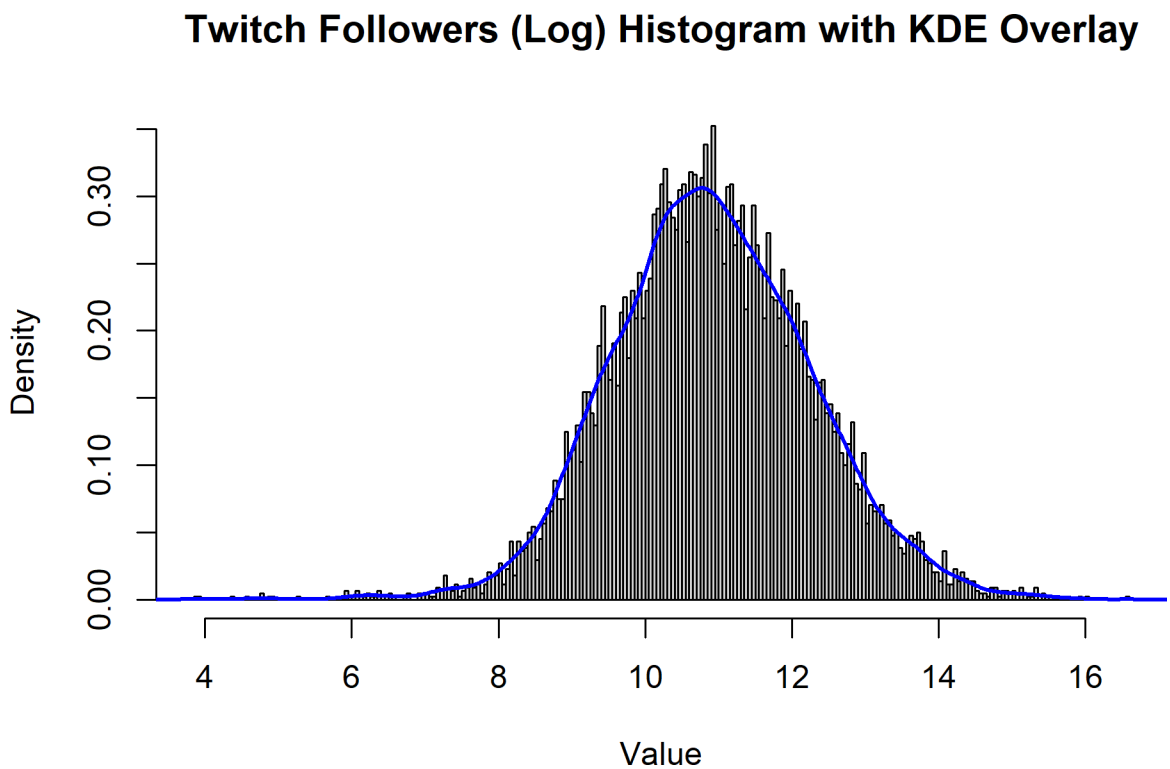


The distribution is very heavily right-skewed, telling us that the majority of Twitch streamers do not have many followers in comparison to the few that have an incredibly large amount. Because of the large number of observations, the values along the axis are very spread out, making it difficult to see the entire histogram. However, the kernel density estimate is capturing the data's underlying structure pretty well.

3.

To assist with seeing the distribution, let's take a log transformation of the data and plot that.

```
foll_dens2 = density(log(twitch$followers), bw = "nrd0", kernel = "gaussian")

hist(log(twitch$followers), breaks = 200, prob = T, border = 'black',
     main = "Twitch Followers (Log) Histogram with KDE Overlay",
     xlab = "Value", ylab = "Density")
lines(foll_dens2, col = "blue", lwd = 2)
```

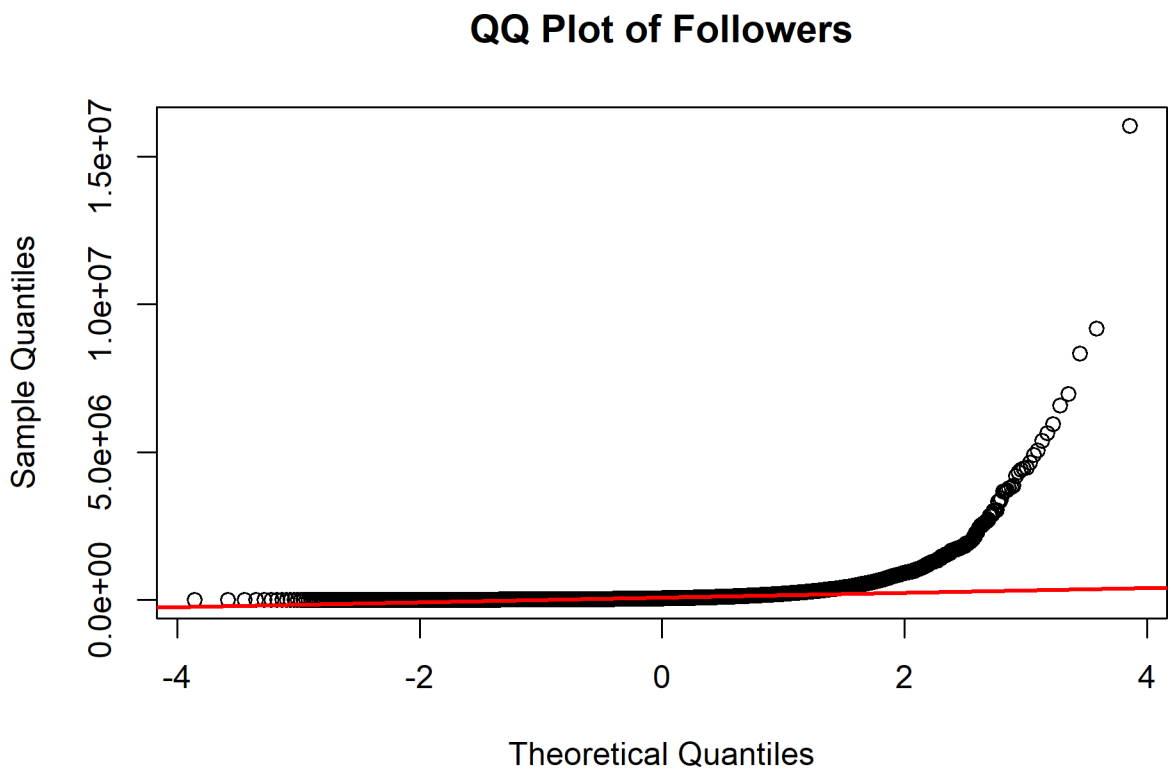


Taking the log transformation improves the look of our distribution. Our data is more spread out, causing the distribution to be much more symmetric. Initially, it was difficult to see the distinctions occurring among those with low follower counts because of the few high-follower count outliers. Again, the kernel density estimate appears to follow the histogram closely.

4.

Based on how the log-transformed data appears, one might assume it follows a normal distribution. This could be tested with a QQ plot as well as a Shapiro-Wilk test (since the test only allows a sample size between 3 and 5000, let's use 5000 random rows from the data). Let's start with the original data.

```
qqnorm(twitch$followers, main = "QQ Plot of Followers")
qqline(twitch$followers, col = "red", lwd = 2)
```

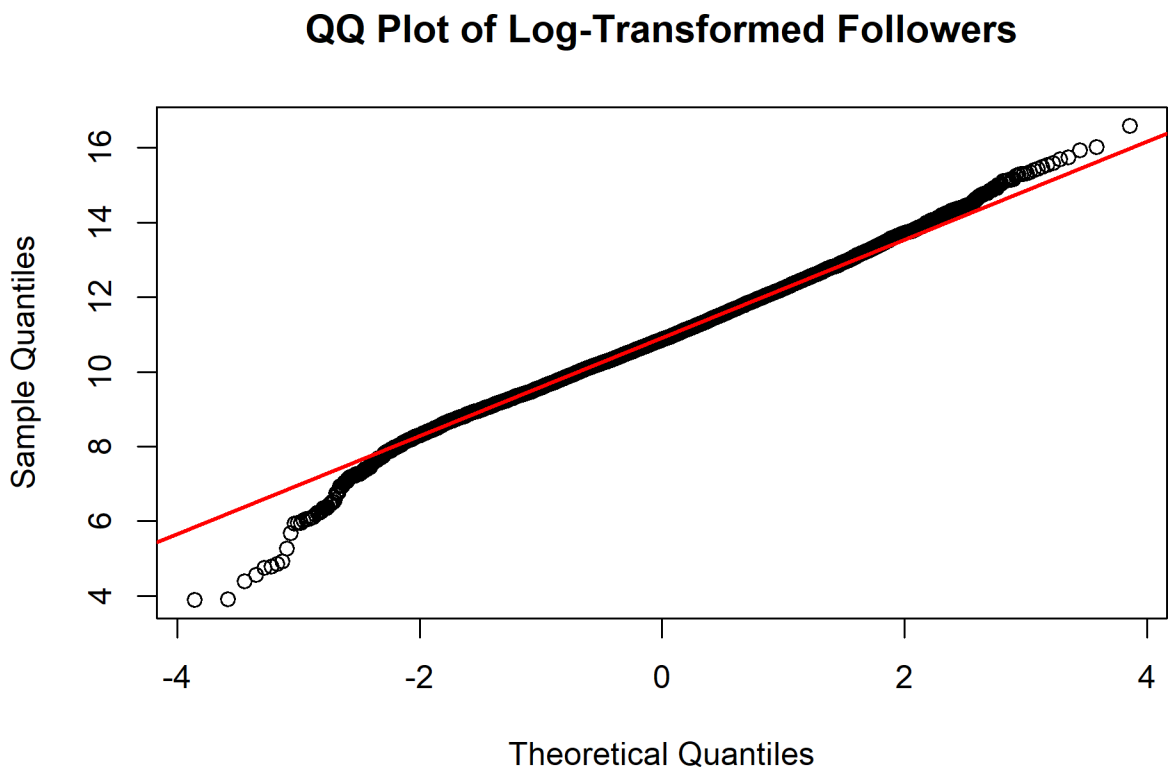



```r
set.seed(123456)
shapiro.test(sample(twitch$followers, 5000))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  sample(twitch$followers, 5000)
## W = 0.26772, p-value < 2.2e-16
```

Based on the QQ Plot and the Shapiro-Wilk test, it appears that our original data is not distributed normally. The QQ plot suggests that our data may be log-normal or even exponential/gamma. The p-value for our normality test indicates that it is certainly not normal. We can check whether it is log-normally distributed by taking the log of our data and plotting that.

```r
qqnorm(log(twitch$followers), main = "QQ Plot of Log-Transformed Followers")
qqline(log(twitch$followers), col = "red", lwd = 2)
```



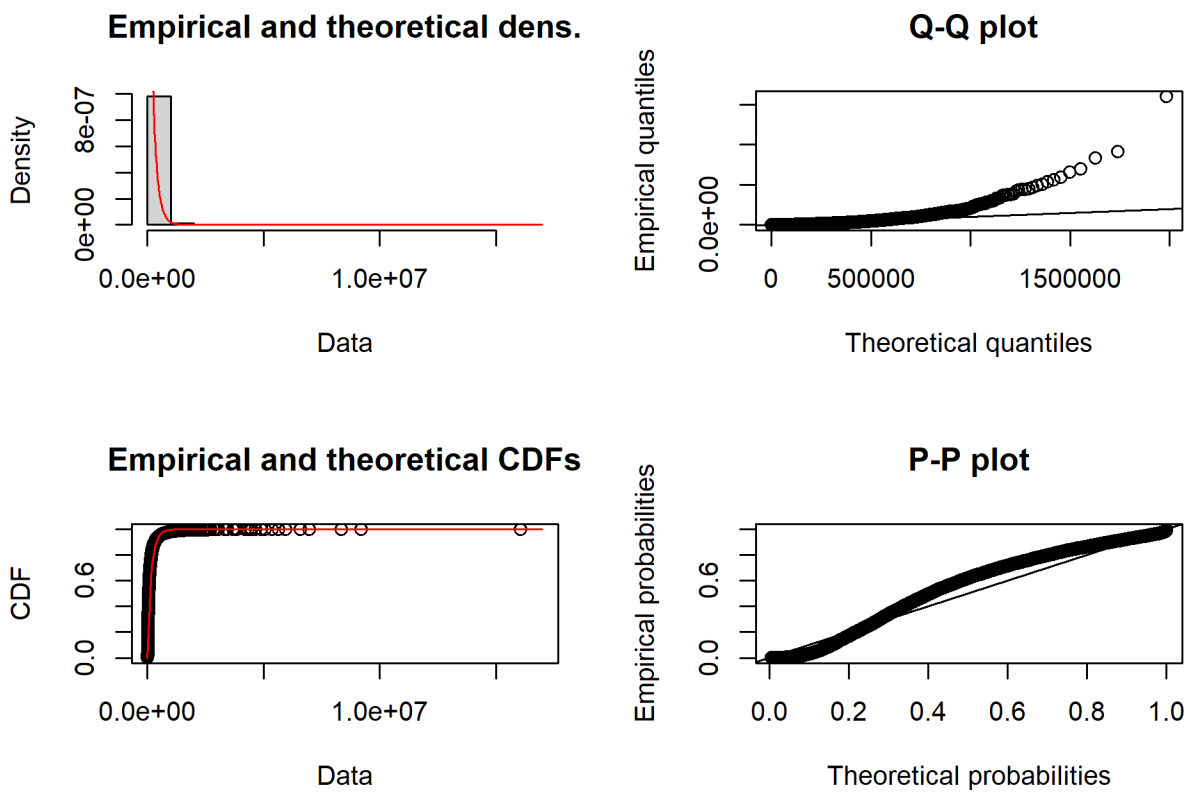

```r
set.seed(123456)
shapiro.test(sample(log(twitch$followers), 5000))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  sample(log(twitch$followers), 5000)
## W = 0.99619, p-value = 4.556e-10
```

The QQ plot suggests that the log of our data might be approximated with a normal distribution; it is mostly a straight line with some curvature at the ends. However, the SW test gives us a very low p-value, telling us even the log-transformed data isn't normally distributed. Interestingly, the p-value did increase from what it was previously, which indicates the normal distribution is a better fit to the transformed data than the original.

Next, let's try a gamma fit:

```r
gamma_fit <- fitdist(twitch$followers, "gamma", method = "mle")

plot(gamma_fit)
```

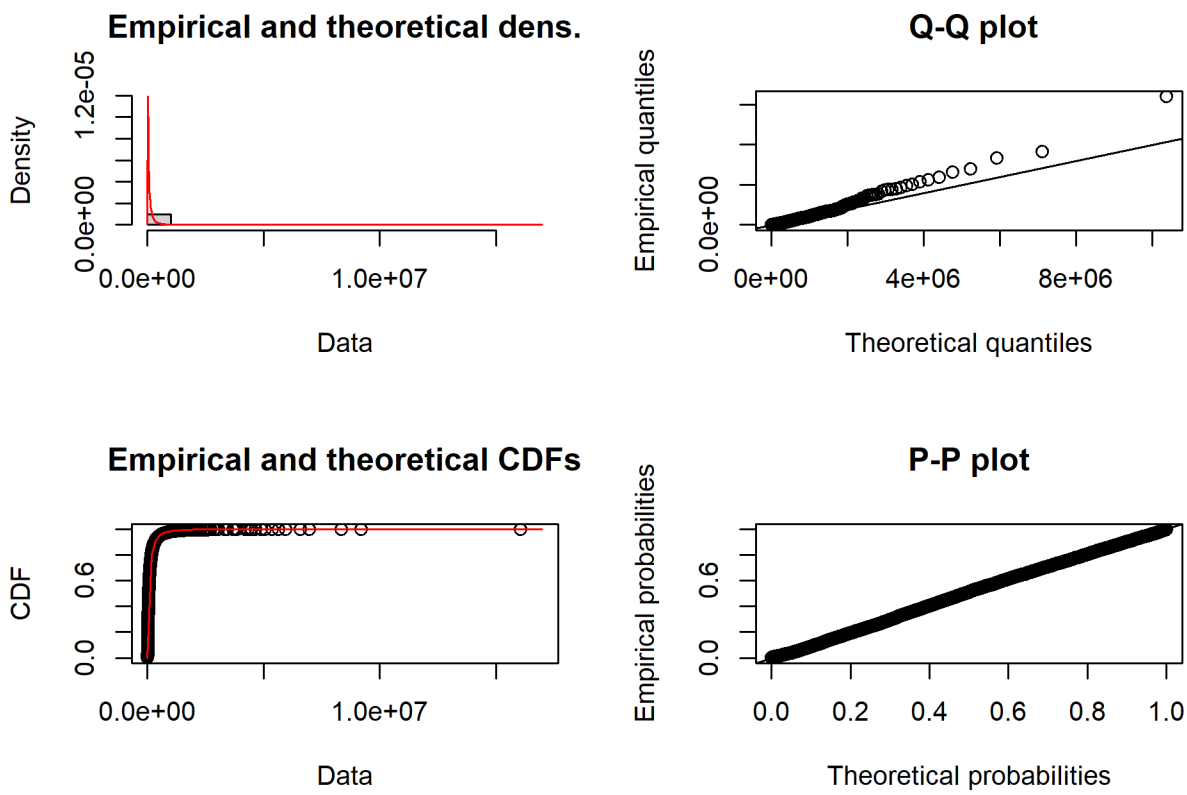



```r
summary(gamma_fit)
```

```
## Fitting of the distribution ' gamma ' by maximum likelihood
## Parameters :
##           estimate   Std. Error
## shape 6.279202e-01 5.802290e-03
## rate  4.324428e-06 4.313497e-08
## Loglikelihood:  -112630.3   AIC:  225264.7   BIC:  225278.8
## Correlation matrix:
##             shape        rate
## shape 1.000000000 0.005999372
## rate  0.005999372 1.000000000
```

We might compare the AIC of the gamma fit to the AIC of the log-normal:

```r
lognorm_fit <- fitdist(twitch$followers, "lnorm", method = "mle")

plot(lognorm_fit)
```




```r
summary(lognorm_fit)
```

```
## Fitting of the distribution ' lnorm ' by maximum likelihood
## Parameters :
##          estimate Std. Error
## meanlog 10.908660 0.01448797
## sdlog    1.359092 0.01024452
## Loglikelihood:  -111182.9   AIC:  222369.7   BIC:  222383.9
## Correlation matrix:
##         meanlog sdlog
## meanlog       1     0
## sdlog         0     1
```

```r
gamma_fit$aic
```

```
## [1] 225264.7
```

```r
lognorm_fit$aic
```

```
## [1] 222369.7
```

Therefore, the AIC regarding the lognormal distribution is lower, indicating the lognormal is a better fit for our original data (and thus the normal distribution is a better fit for the transformed data).

QUESTION 5

1.

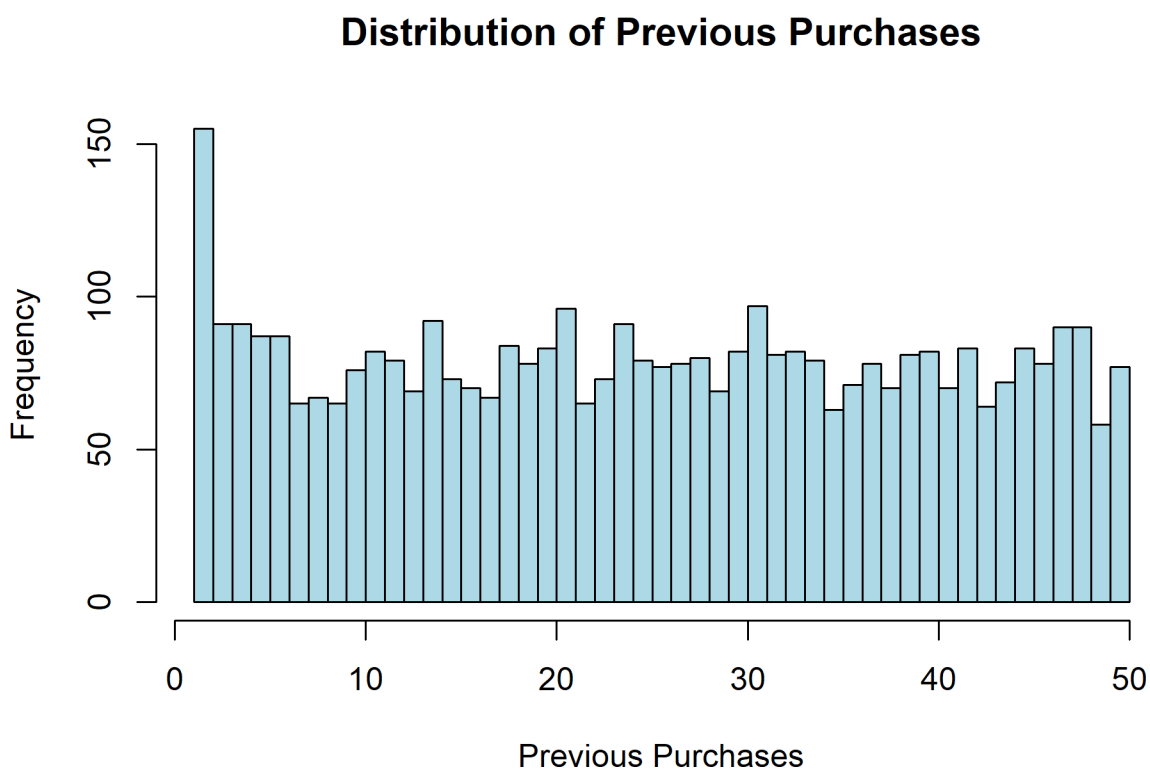
Load the Shopping dataset. Note that you will have to choose it yourself.

```
shop_trends <- read.csv(file.choose(), sep = ',')
```

2.

Let's observe the distribution of the Previous.Purchases column.

```
hist(shop_trends$Previous.Purchases, breaks = 60,
     main = "Distribution of Previous Purchases",
     xlab = "Previous Purchases",
     col = "lightblue",
     border = "black")
```



Now's let's observe the distribution of the Previous.Purchases column under each payment method.

```
unique <- unique(shop_trends$Payment.Method)


par(mfrow = c(ceiling(length(unique) / 2), 2))
for (i in unique) {
  subset <- shop_trends$Previous.Purchases[shop_trends$Payment.Method == i]
  hist(subset,
```

```
        main = paste("Previous Purchases -", i),
        xlab = "Previous Purchases",
        col = "lightgreen",
        border = "black")
}
```

```
payment_counts <- table(shop_trends$Payment.Method)
payment_counts
```

```
##
## Bank Transfer          Cash   Credit Card    Debit Card        PayPal
##           612           670           671           636           677
##         Venmo
##           634
```

```
mean <- tapply(shop_trends$Previous.Purchases, shop_trends$Payment.Method, mean)
mean
```

```
## Bank Transfer          Cash   Credit Card    Debit Card        PayPal
##      24.50163      25.25373      25.58867      25.56132      25.50960
##         Venmo
##      25.64511
```

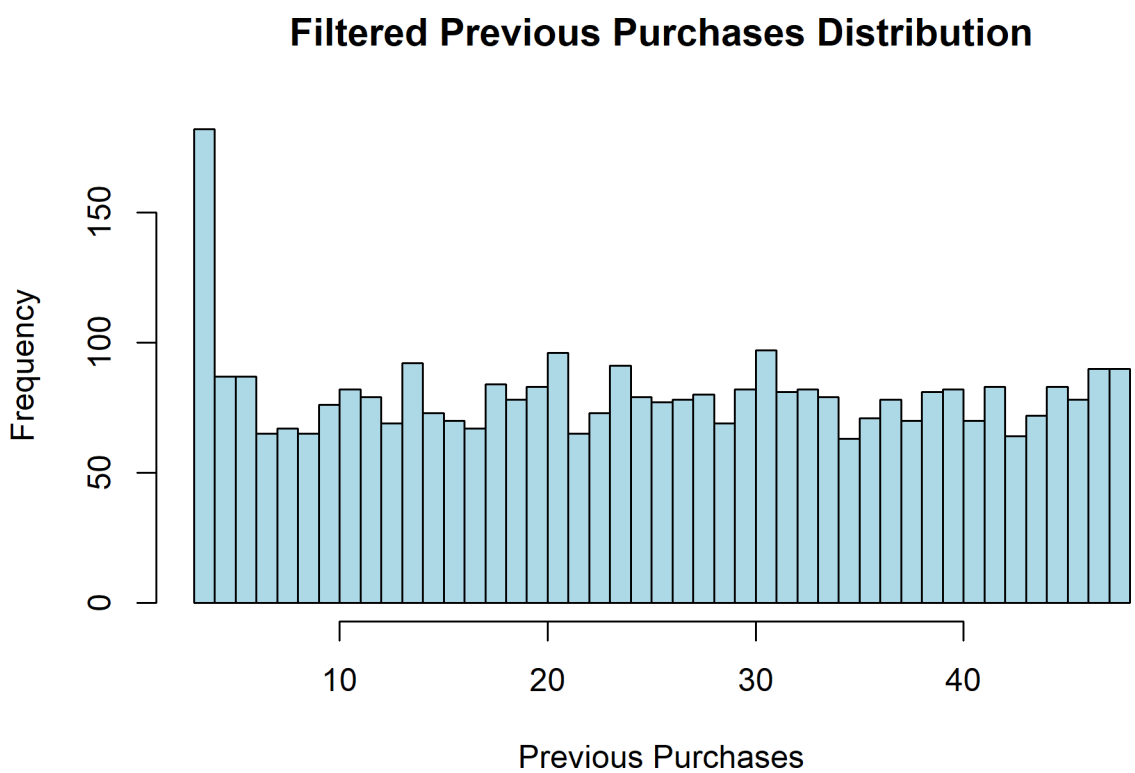
```
median <- tapply(shop_trends$Previous.Purchases, shop_trends$Payment.Method, median)
median
```

```
## Bank Transfer          Cash   Credit Card    Debit Card        PayPal
##            24            25            25            25            26
##         Venmo
##            25
```

The distributions all appear to be very similar. Based on their histograms, means, and medians, each type of payment most likely would not have a great effect on the previous purchases.

Removing the top and bottom 5% of purchases, let's see how the distributions change.

```
lower <- quantile(shop_trends$Previous.Purchases, 0.05, na.rm = TRUE)
upper <- quantile(shop_trends$Previous.Purchases, 0.95, na.rm = TRUE)

new_shop_trends <- shop_trends[shop_trends$Previous.Purchases >= lower &
                               shop_trends$Previous.Purchases <= upper, ]

par(mfrow = c(1,1))
hist(new_shop_trends$Previous.Purchases, breaks = 60,
     main = "Filtered Previous Purchases Distribution",
     xlab = "Previous Purchases",
     col = "lightblue",
     border = "black")
```




```
unique2 <- unique(new_shop_trends$Payment.Method)

par(mfrow = c(ceiling(length(unique2) / 2), 2))
for (i in unique2) {
  subset <- new_shop_trends$Previous.Purchases[new_shop_trends$Payment.Method == i]
  hist(subset,
```

```
        main = paste("Previous Purchases -", i),
        xlab = "Previous Purchases",
        col = "lightgreen",
        border = "black")
}
```

```
payment_counts2 <- table(new_shop_trends$Payment.Method)
payment_counts2
```

```
##
## Bank Transfer          Cash   Credit Card    Debit Card        PayPal
##           566           627           622           579           624
##         Venmo
##           592
```

```
mean2 <- tapply(new_shop_trends$Previous.Purchases,
                new_shop_trends$Payment.Method, mean)
mean2
```

```
## Bank Transfer          Cash   Credit Card    Debit Card        PayPal
##      25.18375      25.57895      25.55949      25.44560      25.69712
##         Venmo
##      25.32770
```

```r
median2 <- tapply(new_shop_trends$Previous.Purchases,
                  new_shop_trends$Payment.Method, median)
median2
```

```
## Bank Transfer          Cash   Credit Card    Debit Card        PayPal
##            25            25            25            25            27
##         Venmo
##            25
```

Removing the top and bottom 5% of purchases does not change much about out data. The histograms all seem to be approximately very similar to how they were before, and the means and median do not change drastically. Therefore, we might conclude that the previous purchases are not dependent on the purchase method.

3.

Now, let's split up our data in training and testing sets to perform regression. First, do so utilizing Purchase.Amount as the dependent variable and training a model with all the other columns as independent factors.

```r
set.seed(123456)
index <- sample(nrow(shop_trends), nrow(shop_trends)*.7)
train <- shop_trends[index,]
test <- shop_trends[-index,]

model <- lm(Purchase.Amount..USD. ~ ., data = train)
summary(model)
```

```
##
## Call:
## lm(formula = Purchase.Amount..USD. ~ ., data = train)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -55.363 -20.178   0.006  20.345  51.925
##
## Coefficients: (4 not defined because of singularities)
##                                      Estimate Std. Error t value Pr(>|t|)
## (Intercept)                         59.8015744  7.6151859   7.853 5.89e-15 ***
## Customer.ID                         -0.0006585  0.0015376  -0.428  0.66849
## Age                                 -0.0150740  0.0309052  -0.488  0.62577
## GenderMale                           0.1433719  2.1231325   0.068  0.94617
## Item.PurchasedBelt                  -1.2369411  3.2957117  -0.375  0.70745
## Item.PurchasedBlouse                 0.3124431  3.3832795   0.092  0.92643
## Item.PurchasedBoots                  3.4886449  3.4105018   1.023  0.30644
## Item.PurchasedCoat                  -0.4180173  3.3285649  -0.126  0.90007
## Item.PurchasedDress                  2.9114764  3.2820803   0.887  0.37512
## Item.PurchasedGloves                 0.4474383  3.4409675   0.130  0.89655
## Item.PurchasedHandbag               -2.3080891  3.3598875  -0.687  0.49217
## Item.PurchasedHat                    1.0181331  3.3771239   0.301  0.76307
## Item.PurchasedHoodie                -0.6258322  3.3849746  -0.185  0.85333
```

```
## Item.PurchasedJacket                -4.0441474  3.3235073  -1.217  0.22378
## Item.PurchasedJeans                  3.5216415  3.5483910   0.992  0.32106
## Item.PurchasedJewelry               -2.9962497  3.2776830  -0.914  0.36073
## Item.PurchasedPants                 -2.5809654  3.3280678  -0.776  0.43811
## Item.PurchasedSandals               -2.1391127  3.3195129  -0.644  0.51937
## Item.PurchasedScarf                  1.1214105  3.3590705   0.334  0.73852
## Item.PurchasedShirt                  1.9425251  3.2189848   0.603  0.54626
## Item.PurchasedShoes                  4.0687849  3.3980243   1.197  0.23126
## Item.PurchasedShorts                 0.4388540  3.4349790   0.128  0.89835
## Item.PurchasedSkirt                 -2.2050040  3.2739596  -0.673  0.50069
## Item.PurchasedSneakers               0.5645818  3.4242654   0.165  0.86905
## Item.PurchasedSocks                 -0.5309999  3.3945280  -0.156  0.87571
## Item.PurchasedSunglasses            -1.2180903  3.3622827  -0.362  0.71717
## Item.PurchasedSweater               -3.2469019  3.3459036  -0.970  0.33193
## Item.PurchasedT-shirt                3.2729858  3.3647254   0.973  0.33078
## CategoryClothing                            NA         NA      NA       NA
## CategoryFootwear                            NA         NA      NA       NA
## CategoryOuterwear                           NA         NA      NA       NA
## LocationAlaska                      11.4072040  4.4931952   2.539  0.01118 *
## LocationArizona                      6.0490791  4.7976162   1.261  0.20748
## LocationArkansas                     0.6847574  4.4068153   0.155  0.87653
## LocationCalifornia                   0.7301500  4.1741934   0.175  0.86116
## LocationColorado                    -1.3398359  4.6265993  -0.290  0.77215
## LocationConnecticut                 -5.6750060  4.4065841  -1.288  0.19791
## LocationDelaware                    -5.1038476  4.3448044  -1.175  0.24022
## LocationFlorida                      1.0040234  4.7447098   0.212  0.83243
## LocationGeorgia                     -0.6329825  4.5795142  -0.138  0.89008
## LocationHawaii                      -1.7131031  4.7050021  -0.364  0.71581
## LocationIdaho                        1.1180319  4.1752074   0.268  0.78889
## LocationIllinois                     1.8722347  4.2640837   0.439  0.66065
## LocationIndiana                      2.3363597  4.4694733   0.523  0.60120
## LocationIowa                         1.8724833  4.8026336   0.390  0.69665
## LocationKansas                      -2.1982910  4.6856207  -0.469  0.63900
## LocationKentucky                    -2.2691903  4.3454790  -0.522  0.60158
## LocationLouisiana                   -0.0699605  4.2632219  -0.016  0.98691
## LocationMaine                       -1.4605380  4.3933440  -0.332  0.73958
## LocationMaryland                    -1.3291452  4.4652596  -0.298  0.76598
## LocationMassachusetts                2.2173143  4.4666632   0.496  0.61964
## LocationMichigan                     6.4626478  4.5028437   1.435  0.15134
## LocationMinnesota                    0.6682292  4.3760934   0.153  0.87865
## LocationMississippi                  0.9847462  4.3801269   0.225  0.82214
## LocationMissouri                     0.0156338  4.4801611   0.003  0.99722
## LocationMontana                      1.1739601  4.2329132   0.277  0.78154
## LocationNebraska                     3.0873683  4.2107498   0.733  0.46350
## LocationNevada                       5.9393601  4.3207905   1.375  0.16937
## LocationNew Hampshire                2.4285470  4.5659594   0.532  0.59485
## LocationNew Jersey                   0.8507003  4.7359704   0.180  0.85746
## LocationNew Mexico                   4.9271748  4.4436675   1.109  0.26762
## LocationNew York                     2.2537853  4.3410034   0.519  0.60368
## LocationNorth Carolina               5.7541873  4.5019798   1.278  0.20131
## LocationNorth Dakota                 5.7681175  4.4078387   1.309  0.19078
## LocationOhio                         1.0866250  4.3490920   0.250  0.80272
## LocationOklahoma                     1.5820617  4.4609177   0.355  0.72288
## LocationOregon                      -0.2664007  4.5640491  -0.058  0.95346
```

```
## LocationPennsylvania                10.5850973  4.7088477   2.248  0.02466 *
## LocationRhode Island                 2.5622808  4.8601028   0.527  0.59809
## LocationSouth Carolina               3.1255363  4.5881986   0.681  0.49580
## LocationSouth Dakota                 1.0707062  4.6798514   0.229  0.81905
## LocationTennessee                    3.7494870  4.4399223   0.844  0.39847
## LocationTexas                        2.1793469  4.4476973   0.490  0.62418
## LocationUtah                         2.8822618  4.5784146   0.630  0.52906
## LocationVermont                     -0.4165252  4.4489149  -0.094  0.92542
## LocationVirginia                     4.0353940  4.4071921   0.916  0.35994
## LocationWashington                   8.3113591  4.6420961   1.790  0.07350 .
## LocationWest Virginia                4.8351181  4.4207636   1.094  0.27417
## LocationWisconsin                   -0.2894824  4.4472819  -0.065  0.94811
## LocationWyoming                      3.9644304  4.4205572   0.897  0.36990
## SizeM                                1.2482191  1.1328843   1.102  0.27065
## SizeS                                2.8822386  1.4467777   1.992  0.04646 *
## SizeXL                              -0.4694126  1.6571018  -0.283  0.77699
## ColorBlack                           0.1286928  3.1802044   0.040  0.96772
## ColorBlue                           -1.1466828  3.2703394  -0.351  0.72589
## ColorBrown                           1.0728279  3.3578947   0.319  0.74938
## ColorCharcoal                        2.5126720  3.3824742   0.743  0.45764
## ColorCyan                            2.8414604  3.3010676   0.861  0.38944
## ColorGold                            1.8703913  3.3475488   0.559  0.57639
## ColorGray                            3.3704847  3.3318949   1.012  0.31183
## ColorGreen                           7.1188970  3.2078096   2.219  0.02656 *
## ColorIndigo                         -4.3350484  3.3505624  -1.294  0.19584
## ColorLavender                        1.5903399  3.3942319   0.469  0.63944
## ColorMagenta                        -1.9894327  3.3850416  -0.588  0.55678
## ColorMaroon                         -1.2063854  3.2530954  -0.371  0.71079
## ColorOlive                          -1.1574873  3.1395990  -0.369  0.71240
## ColorOrange                          0.4074237  3.2366894   0.126  0.89984
## ColorPeach                           1.0216275  3.3126730   0.308  0.75780
## ColorPink                            0.0684996  3.3332291   0.021  0.98361
## ColorPurple                          0.6199591  3.3455442   0.185  0.85300
## ColorRed                            -1.3378649  3.2601378  -0.410  0.68157
## ColorSilver                         -2.8826311  3.2180807  -0.896  0.37046
## ColorTeal                            0.8110312  3.2541476   0.249  0.80320
## ColorTurquoise                      -2.9464483  3.3424756  -0.882  0.37812
## ColorViolet                          3.7879986  3.2263368   1.174  0.24047
## ColorWhite                           4.4559061  3.4049458   1.309  0.19077
## ColorYellow                          0.6986978  3.2275507   0.216  0.82863
## SeasonSpring                        -3.4352709  1.3254923  -2.592  0.00960 **
## SeasonSummer                        -3.6406086  1.3165858  -2.765  0.00573 **
## SeasonWinter                        -2.5936708  1.3143057  -1.973  0.04855 *
## Review.Rating                        0.7363478  0.6558431   1.123  0.26165
## Subscription.StatusYes              -0.5696416  1.9493303  -0.292  0.77014
## Shipping.TypeExpress                -0.7516362  1.6260539  -0.462  0.64394
## Shipping.TypeFree Shipping          -1.3088192  1.6049439  -0.815  0.41487
## Shipping.TypeNext Day Air           -1.2659889  1.6158178  -0.783  0.43341
## Shipping.TypeStandard               -3.1924183  1.6134573  -1.979  0.04796 *
## Shipping.TypeStore Pickup           -1.1410520  1.6171037  -0.706  0.48049
## Discount.AppliedYes                 -2.3543551  1.9355243  -1.216  0.22395
## Promo.Code.UsedYes                          NA         NA      NA       NA
## Previous.Purchases                   0.0096077  0.0325388   0.295  0.76781
## Payment.MethodCash                   1.4049273  1.6019930   0.877  0.38057
```

```
## Payment.MethodCredit Card             0.0469542  1.6292111   0.029  0.97701
## Payment.MethodDebit Card              2.5040500  1.6641244   1.505  0.13252
## Payment.MethodPayPal                 -0.2359807  1.6194269  -0.146  0.88415
## Payment.MethodVenmo                  -0.1334574  1.6334925  -0.082  0.93489
## Frequency.of.PurchasesBi-Weekly       1.3091931  1.7389015   0.753  0.45159
## Frequency.of.PurchasesEvery 3 Months  0.4639257  1.7155758   0.270  0.78686
## Frequency.of.PurchasesFortnightly    -0.6058752  1.7290812  -0.350  0.72606
## Frequency.of.PurchasesMonthly         0.5433453  1.6804083   0.323  0.74646
## Frequency.of.PurchasesQuarterly      -0.8443426  1.7171578  -0.492  0.62297
## Frequency.of.PurchasesWeekly         -1.2988333  1.7112961  -0.759  0.44794
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 23.77 on 2603 degrees of freedom
## Multiple R-squared:  0.0493, Adjusted R-squared:  0.003281
## F-statistic: 1.071 on 126 and 2603 DF,  p-value: 0.2816
```

The R-squared of our model with all the variables is approximately 4.93%, which is extremely low. Moreover, the adjusted R-squared is .3281%; therefore, we might conclude that many of these independent variables do not help much in explaining the variation in our model. The F-stat is also not large and it's p-value is .2816, telling us that this model is not an improvement compared to a model with no predictor variables.

```r
predict <- predict(model, test)
rmse <- sqrt(mean((predict - test$Purchase.Amount..USD.)^2))
rmse
```

```
## [1] 23.69124
```

The root mean squared error evaluated on the testing data is 23.69124. Let's see how this value changes if we test the model simply using two variables as the independent ones.

```r
model2 <- lm(Purchase.Amount..USD. ~ Review.Rating + Previous.Purchases, data = train)
summary(model2)
```

```
##
## Call:
## lm(formula = Purchase.Amount..USD. ~ Review.Rating + Previous.Purchases,
##     data = train)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -40.818 -21.447   0.331  20.805  40.800
##
## Coefficients:
##                     Estimate Std. Error t value Pr(>|t|)
## (Intercept)        57.454997   2.549973  22.532   <2e-16 ***
## Review.Rating       0.609918   0.638673   0.955    0.340
## Previous.Purchases  0.009363   0.031684   0.296    0.768
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 23.82 on 2727 degrees of freedom
## Multiple R-squared:  0.0003698,  Adjusted R-squared:  -0.0003634
## F-statistic: 0.5044 on 2 and 2727 DF,  p-value: 0.6039
```

The R-squared of this reduced model is .03698%, much less than what we had with the previous model. The adjusted R-squared is even negative, which further confirms that the "explanatory" variables in our model do not explain very much of anything. The F-statistic is not significant at all, with a high p-value of .6039, telling us that the variables we kept are not a major improvement compared to a model with no variables.

```
predict2 <- predict(model2, test)
rmse2 <- sqrt(mean((predict2 - test$Purchase.Amount..USD.)^2))
rmse2
```

```
## [1] 23.36914
```

The RMSE under this reduced model is 23.36914, which is just slightly less than what it was previously. This tells us that all the variables we eliminated do not contribute significantly to the improvement of our model. As could be observed previously, many of them did not have statistical significance anyway, so this is not a great surprise.